

ระบบตรวจสอบความมีชีวิต (Liveness) และยืนยันใบหน้า แบบทำงานบนอุปกรณ์ทั้งหมด

eKYC Local — ไม่เชื่อมต่อเซิร์ฟเวอร์ ไม่ส่งข้อมูลชีวมิติออกจากเครื่อง

ชื่อระบบ	eKYC Local (แพ็คเกจ <code>@ekyc/react-native-ekyc-local</code> และแอป <code>apps/local</code>)
เวอร์ชันเอกสาร	1.0
วันที่จัดทำ	19 สิงหาคม 2569
สถานะ	ฉบับส่งมอบเพื่อทดสอบภายใน (ไม่ใช่เอกสารรับรองมาตรฐาน)
แหล่งโค้ด	https://github.com/watcharaponthod-code/ekyc
ไฟล์ติดตั้ง	GitHub Releases ของโครงการ (ไฟล์ <code>ekyc-local-*.apk</code>)

เอกสารฉบับนี้อธิบายการออกแบบ ขอบเขต ผลการทดสอบอ้างอิง และข้อจำกัดของระบบตามสภาพจริง ณ วันที่จัดทำ ตัวเลขทุกค่าที่ปรากฏเป็นค่าที่วัดได้ในเงื่อนไขที่ระบุไว้ ไม่ใช่การรับรองสมรรถนะโดยหน่วยงานภายนอก

สารบัญ

1. บทนำและวัตถุประสงค์

2. ขอบเขตของระบบและสิ่งที่ระบบไม่ครอบคลุม

3. สถาปัตยกรรมและองค์ประกอบ

4. ขั้นตอนการทำงาน

5. รายละเอียดทางเทคนิค

5.1 การตรวจจับใบหน้าและการไค้ช (ML Kit)

5.2 การฝังใบหน้า (MobileFaceNet TFLite)

5.3 เกณฑ์การตัดสินใจและการเปรียบเทียบ

6. การทดสอบและรูปร่าง

6.1 การทดสอบอัตโนมัติ

6.2 การเปรียบเทียบโมเดลบนชุดข้อมูลสาธารณะ

6.3 หน้าจอของแอปพลิเคชัน

7. การติดตั้งและการทดสอบด้วยตนเอง

8. ความปลอดภัยและข้อมูลส่วนบุคคล

9. ข้อจำกัดและแผนงานถัดไป

ภาคผนวก ก. รหัสเหตุผล (reason codes)

ภาคผนวก ข. โครงสร้างไฟล์ที่เกี่ยวข้อง

1. บทนำและวัตถุประสงค์

ระบบ eKYC Local เป็นส่วนประกอบ (module) สำหรับแอปพลิเคชัน React Native ที่ทำหน้าที่ตรวจสอบว่าบุคคลที่อยู่หน้ากล้องกำลังปฏิบัติตามคำสั่งจริงในขณะนั้น (หันซ้าย หันขวา อ้าปาก พยักหน้า) และ ใบหน้าที่ปรากฏในทุกท่าเป็นบุคคลเดียวกัน โดยการประมวลผลทั้งหมดเกิดขึ้นบนอุปกรณ์ ไม่มีการเชื่อมต่อเครือข่าย ไม่มีเซิร์ฟเวอร์ และไม่มีข้อมูลชีวมิติ (ภาพหรือแม่แบบใบหน้า) ออกจากเครื่อง

ระบบนี้จัดทำแยกจากระบบ eKYC แบบ client-server ของโครงการ (ซึ่งมีเซิร์ฟเวอร์เป็นผู้ตัดสินใจ) ตามข้อกำหนดที่ต้องการทางเลือกที่ทำงานได้เองทั้งหมดบนโทรศัพท์ ด้วยเทคโนโลยีคอมพิวเตอร์วิทัศน์ของ Google (ML Kit) และโมเดลจดจำใบหน้าขนาดเล็กที่ทำงานบนอุปกรณ์ได้ (MobileFaceNet)

วัตถุประสงค์

1. ยืนยันการมีชีวิตแบบตอบสนองต่อคำสั่ง (active liveness) โดยไม่พึ่งพาเซิร์ฟเวอร์
2. ยืนยันความสอดคล้องของอัตลักษณ์ตลอดการทำท่า (identity consistency) เพื่อป้องกันการสลับบุคคลหรือสลับภาพระหว่างขั้นตอน
3. รองรับการบันทึกแม่แบบใบหน้าไว้ในเครื่องเพื่อยืนยันตัวบุคคลเดิมในครั้งถัดไป (local enrol / verify) โดยเป็นทางเลือก
4. ให้ตัวเลขผลการทดสอบทุกครั้งแสดงบนหน้าจอ เพื่อให้ผู้ทดสอบปรับเกณฑ์กับผู้ใช้จริงได้

2. ขอบเขตของระบบและสิ่งที่ระบบไม่ครอบคลุม

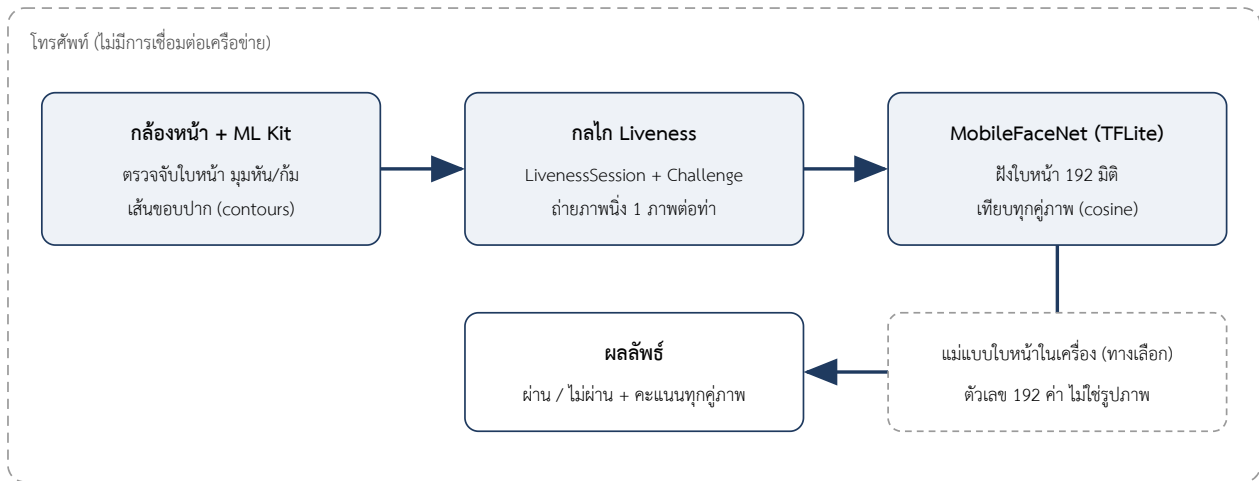
ตารางที่ 1 ขอบเขตของระบบ

รายการ	สถานะ
การโค้ชผู้ใช้ให้อยู่ในกรอบ (ไกล/ใกล้/เยื้อง/หลายใบหน้า)	ครอบคลุม — ใช้กลไกเดียวกับระบบ client-server
คำสั่งท่าทาง: หันซ้าย หันขวา อ้าปาก พยักหน้า (สุ่มลำดับทุกครั้ง)	ครอบคลุม
การตรวจว่าทุกภาพเป็นบุคคลเดียวกัน	ครอบคลุม — MobileFaceNet เทียบทุกคู่ภาพ
การบันทึก/ยืนยันกับแม่แบบใบหน้าในเครื่อง	ครอบคลุม (ทางเลือก)
หน้ากากแข็ง (3D-print, resin, กระจก)	ครอบคลุมโดยอ้อม — คำสั่ง “อ้าปาก” ต้องผ่านภายในเวลาที่กำหนด หน้ากากแข็งอาจขกรไกรไม่ได้
หน้ากากซิลิโคน / latex	มีการวัดชีพจรจากสีผิว (rPPG) หลังจบคำสั่ง — รายงานคะแนน แต่ยังไม่ตัดสิน (advisory) จนกว่าจะปรับเทียบบนโทรศัพท์จริง
การตรวจจับภาพถ่าย จอภาพ วิดีโอย้อนหลัง (2D PAD)	ไม่ครอบคลุม — ไม่มีโมเดล PAD 2 มิติบนอุปกรณ์ในรุ่นนี้
การเชื่อมต่อเซิร์ฟเวอร์ การส่งข้อมูล การเก็บบันทึกกลาง	ไม่มีโดยเจตนา

ข้อควรระวังในการนำไปใช้: เนื่องจากไม่มีการตรวจสอบ 2 มิติ วิดีโอของบุคคลจริงที่เล่นบนจอและทำตามคำสั่งอาจผ่านการตรวจได้ (rPPG อ่านชีพจรจากวิดีโอของคนจริงบนจอได้เช่นกัน) ระบบนี้จึงเหมาะสำหรับงานยืนยันซ้ำที่มีความเสี่ยงต่ำ (เช่น ยืนยันว่ายังเป็นผู้ใช้คนเดิมก่อนทำรายการภายในแอป) ไม่เหมาะเป็นหลักฐานเดียวสำหรับการเปิดบัญชีหรือการพิสูจน์ตัวตนระดับสูง ซึ่งควรใช้ระบบ client-server ที่มีชั้นป้องกันบนเซิร์ฟเวอร์

3. สถาปัตยกรรมและองค์ประกอบ

ระบบประกอบด้วยสามส่วนบนอุปกรณ์เดียว ดังรูปที่ 1



รูปที่ 1 องค์ประกอบของระบบ eKYC Local ทั้งหมดอยู่บนอุปกรณ์

ตารางที่ 2 องค์ประกอบซอฟต์แวร์

องค์ประกอบ	เทคโนโลยี	บทบาท
กล้อง	react-native-vision-camera 5	ภาพตัวอย่างและการจับภาพนิ่งจากพื้นผิวแสดงผล (ไม่มีความหน่วงชัดเตอร์)
ตรวจจับใบหน้า	Google ML Kit Face Detection ผ่าน react-native-vision-camera-face-detector 2	มุมหัน (yaw) มุมก้ม (pitch) มุมเอียง (roll) ความน่าจะเป็นตาเปิดและเส้นขอบริมฝีปาก
กลไก liveness	@ekyc/react-native-ekyc (LivenessSession, Challenge, UI)	ลำดับคำสั่ง การถือท่า การเลือกจังหวะถ่ายภาพ ข้อความภาษาไทย/อังกฤษ
ฝังใบหน้า	MobileFaceNet (TensorFlow Lite, 5.2 MB) ผ่าน react-native-fast-tflite	เวกเตอร์ 192 มิติต่อภาพ
เตรียมภาพ	expo-image-manipulator + jpeg-js	ตัดกรอบ หมุนให้ตาอยู่ระดับ ย่อเป็น 112×112 พิกเซล ถอดรหัสเป็นพิกเซล

ที่เก็บแม่แบบ	expo-file-system (พื้นที่ส่วนตัวของแอป)	ไฟล์ JSON ตัวเลข 192 ค่า
---------------	---	--------------------------

4. ขั้นตอนการทำงาน

1. **เริ่มต้น** — แอปโหลดโมเดล MobileFaceNet ล่วงหน้าตั้งแต่หน้าแรก (แสดงสถานะ “MobileFaceNet ready” พร้อมรูปร่างเทนเซอร์เข้า/ออก)
2. **จัดกรอบ** — ระบบแนะนำให้ผู้ใช้อยู่ในกรอบวงรี (ขยับเข้า/ออก/กึ่งกลาง) และต้องมีใบหน้าเดียว
3. **ทำมองตรง (center)** — เมื่อ $|yaw|$ และ $|pitch| \leq 12^\circ$ ค้าง 400 ms ระบบถ่ายภาพ **neutral**
4. **คำสั่งลุ่มลำดับ 4 ทำ** — หันซ้าย/หันขวา ($|yaw| \geq 18^\circ$), อ้าปาก (อัตราส่วนช่องว่างริมฝีปากต่อความกว้างปาก ≥ 0.30), พยักหน้า ($|pitch| \geq 15^\circ$) แต่ละท่าถ่ายภาพเมื่อถือทำได้ครั้งหนึ่งของเวลาที่กำหนด แต่ละขั้นมีเวลาไม่เกิน 12 วินาที ทั้งหมดไม่เกิน 60 วินาที
5. **อยู่นิ่งวัดชีพจร (rPPG)** — หลังคำสั่งครบ แอปแสดง “อยู่นิ่งๆ มองกล้อง สักครู” และถ่ายภาพต่อเนื่องประมาณ 7 วินาที (เร็วเท่าที่อุปกรณ์ทำได้ ราว 8–15 ภาพ/วินาที) จากนั้นอ่านสีเฉลี่ยของหน้าผากและแก้มสองข้างจากภาพย่อ 32×32 ของกรอบใบหน้าในแต่ละภาพ และวิเคราะห์หาความถี่ชีพจร (0.7–3 Hz) ด้วยวิธี POS + สเปกตรัมเฉลี่ยข้าม 3 บริเวณ (โค้ดเดียวกับฝั่งเซิร์ฟเวอร์ แปลงเป็น TypeScript)
6. **ตรวจซ้ำบนภาพนิ่ง** — ML Kit ตรวจจับใบหน้าบนไฟล์ภาพแต่ละภาพอีกครั้ง (โหมด accurate) เพื่อให้ได้กรอบและมุมเอียงที่ตรงกับภาพจริง
7. **ฝังใบหน้า** — ตัดกรอบใบหน้าขยาย 15 % หมุนชดเชยมุมเอียง ตัดกลาง ย่อ $112 \times 112 \rightarrow$ MobileFaceNet $\rightarrow$ เวกเตอร์หน่วย
8. **ตัดสิน** — คำนวณ cosine similarity ทุกคู่ภาพ ผ่านเมื่อค่าต่ำสุดของทุกคู่ ≥ 0.45 (และเมื่อเลือกโหมด verify: ภาพ neutral ต้องคล้ายแม่แบบที่บันทึกไว้ ≥ 0.55)
9. **แสดงผล** — ผ่าน/ไม่ผ่าน พร้อมรหัสเหตุผล ลำดับท่า คะแนนทุกคู่ภาพ ผลชีพจร (คะแนน, bpm, ความเด่นของยอด, จำนวนภาพ, อัตราภาพ) และเวลาที่ใช้

การเลือกจังหวะถ่ายภาพบนโทรศัพท์เป็นเพียงการ “เลือกช่วงเวลา” เท่านั้น การตัดสินอัตลักษณ์ทำจากพิกเซลของภาพนิ่งที่ถูกตรวจซ้ำ ไม่ได้เชื่อค่าจากภาพตัวอย่างสด

5. รายละเอียดทางเทคนิค

5.1 การตรวจจับใบหน้าและการไค้ช (ML Kit)

ตัวตรวจจับทำงานในโหมด fast พร้อม classification, landmarks และ contours ค่าที่ใช้ต่อเฟรม ได้แก่ จำนวน ใบหน้า, yaw, pitch, roll, ความน่าจะเป็นตาซ้าย/ขวาเปิด, ความน่าจะเป็นยิ้ม, กรอบใบหน้า (normalised 0–1) และ **ระดับการอ้าปาก** ซึ่งคำนวณจากเส้นขอบ UPPER_LIP_BOTTOM และ LOWER_LIP_TOP เป็นอัตราส่วนช่องว่างต่อความกว้างปาก (ปิด ≈ 0 , อ้าชัดเจน 0.3–0.6 ไม่ขึ้นกับระยะกล้อง) หากไม่มี contours จะใช้ระยะฐานจมูก–ใต้ปากต่อความกว้างปากเป็นตัวแทน (มีการทดสอบหน่วยครอบคลุมทั้งสองเส้นทาง)

เครื่องหมายทิศทางของ yaw/pitch ขึ้นกับอุปกรณ์ จึงออกแบบให้คำสั่ง “พยักหน้า” และการตรวจ “หันคนละทาง” ไม่ผูกกับเครื่องหมาย (ต้องหันครบทั้งซ้ายและขวาในเซสชันเดียวกัน)

5.2 การฝังใบหน้า (MobileFaceNet TFLite)

ไฟล์โมเดล	<code>packages/react-native-ekyc-local/assets/mobile_face_net.tflite</code> ขนาด 5,243,108 ไบต์
เทนเซอร์เข้า	[1, 112, 112, 3] float32, ปรับค่าเป็น (พิกเซล – 127.5) / 128, ลำดับ RGB
เทนเซอร์ออก	[1, 192] float32 (ตรวจสอบด้วย LiteRT interpreter: ค่า norm ของเวกเตอร์ออก = 1.0)
การเตรียมภาพ	กรอบ ML Kit ขยาย 15 % → สี่เหลี่ยมจัตุรัสด้าน v2 เท่าเพื่อกันมุมดำเมื่อหมุน → หมุน –roll → ตัดกลาง → ย่อ 112×112 → JPEG q92 → ถอดรหัสด้วย jpeg-js
เวลาโดยประมาณ	30–60 ms ต่อใบหน้านับโทรศัพท์ระดับกลาง (แสดงจริงบนหน้าผลลัพธ์)

5.3 เกณฑ์การตัดสินใจและการปรับเทียบ

ตารางที่ 3 เกณฑ์ที่ใช้และที่มา

เกณฑ์	ค่าเริ่มต้น	ความหมายและที่มา
-------	-------------	------------------

consistencyMin	0.45	ค่าคล้ายคลึงต่ำสุดในทุกคู่ภาพของเซสชันเดียวกัน จากการเปรียบเทียบบน LFW: คู่คนละคน 6.5 % ผ่านเกณฑ์นี้ ขณะที่ภาพในเซสชันเดียวกัน (แสงเดียวกัน ห่างกันไม่กวีนาที) มีค่าสูงกว่าคู่ LFW ที่ต่างปี/ต่างมุมมาก
matchMin	0.55	ค่าคล้ายคลึงระหว่างภาพ neutral กับแม่แบบที่บันทึกไว้ (ข้ามเซสชัน) เข้มกว่า: คู่คนละคนบน LFW ผ่าน 1.4 %
ท่าหัน	18°	yaw บนโทรศัพท์ (ML Kit)
พยักหน้า	15°	pitch
อ้าปาก	0.30	อัตราส่วนช่องว่างริมฝีปาก/ความกว้างปาก
pulseRule / pulseMin / pulseMs	advisory / 0.5 / 7000	ชั้น rPPG: advisory รายงานคะแนนโดยไม่ตัดสิน, enforce เพิ่มรหัส PULSE_ABSENT เมื่อคะแนน < pulseMin, off ข้ามขั้นตอน; คะแนน 0.5 = ความเด่นของยอด 7 dB (เปรียบเทียบบนสัญญาณสังเคราะห์: แยกชัดเมื่อแอมพลิจูดชีพจร $\geq \sim 2\times$ สัญญาณรบกวน)

ค่าทั้งหมดปรับได้ผ่าน props ของ **LocalLivenessCamera** และควรวัดซ้ำกับกลุ่มผู้ใช้จริงก่อนใช้งานจริง (ดูหัวข้อ 6.2)

6. การทดสอบและรูปร่าง

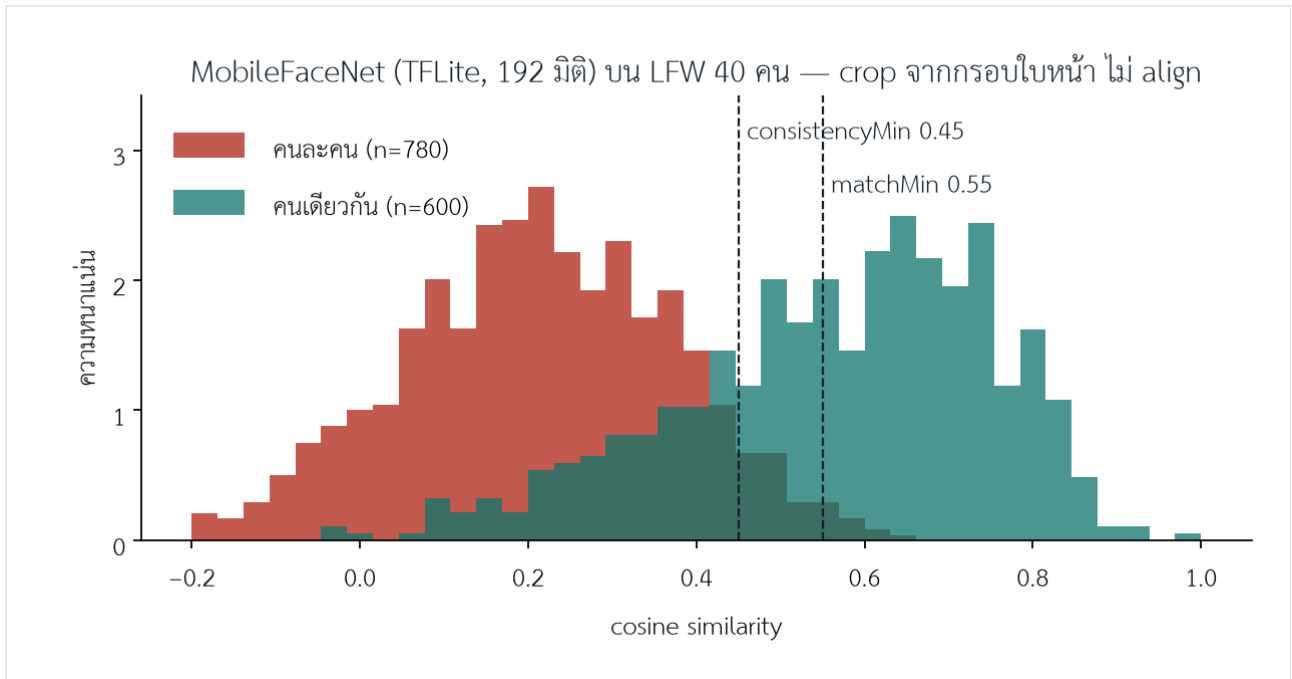
6.1 การทดสอบอัตโนมัติ

ตารางที่ 4 ชุดทดสอบอัตโนมัติที่เกี่ยวข้อง (ผลล่าสุด 18 ส.ค. 2569)

ชุดทดสอบ	ผล	ครอบคลุม
<code>packages/react-native-ekyc-local</code> (jest)	29 ผ่าน / 29	cosine, การปรับค่าพิกเซล, ถอดรหัส JPEG + base64, การหาจุดที่แย่มากที่สุด, การตัดสินใจ (สลับภาพ/คนละคน/แม่แบบอ้างอิง), การสุ่มลำดับทำ, NodChallenge, rPPG (สัญญาณจริง/หน้ากาก/drift/ภาพนิ่ง/fail-closed/8-30 fps), การอ่านสปีรียูนิค
<code>packages/react-native-ekyc</code> (jest)	105 ผ่าน / 105	LivenessSession, ทุก Challenge รวมอ้าปากและพยักหน้า, mouthOpenness จาก contours, ข้อความ UI ไทย/อังกฤษครบทุกรหัส
TypeScript typecheck	ผ่าน	ทั้งสองแพ็คเกจและแอป <code>apps/local</code>

6.2 การเปรียบเทียบโมเดลบนชุดข้อมูลสาธารณะ

เพื่อยืนยันว่าโมเดล MobileFaceNet และขั้นตอนเตรียมภาพทำงานได้จริง ได้ทดสอบกับชุดข้อมูล LFW (Labeled Faces in the Wild) จำนวน 40 บุคคล ภาพละ 6 ภาพ โดยตัดกรอบใบหน้าจากตำแหน่งกลางภาพโดยไม่จัดแนว (สภาพที่ยากกว่าการใช้งานจริงซึ่งภาพมาจากเซสชันเดียวกัน)



รูปที่ 2 การกระจายค่า cosine similarity ระหว่างคู่ภาพคนเดียวกัน (n=600) และคนละคน (n=780) พร้อมเกณฑ์ที่ใช้

ตารางที่ 5 สถิติจากการเปรียบเทียบ (LFW, 40 คน)

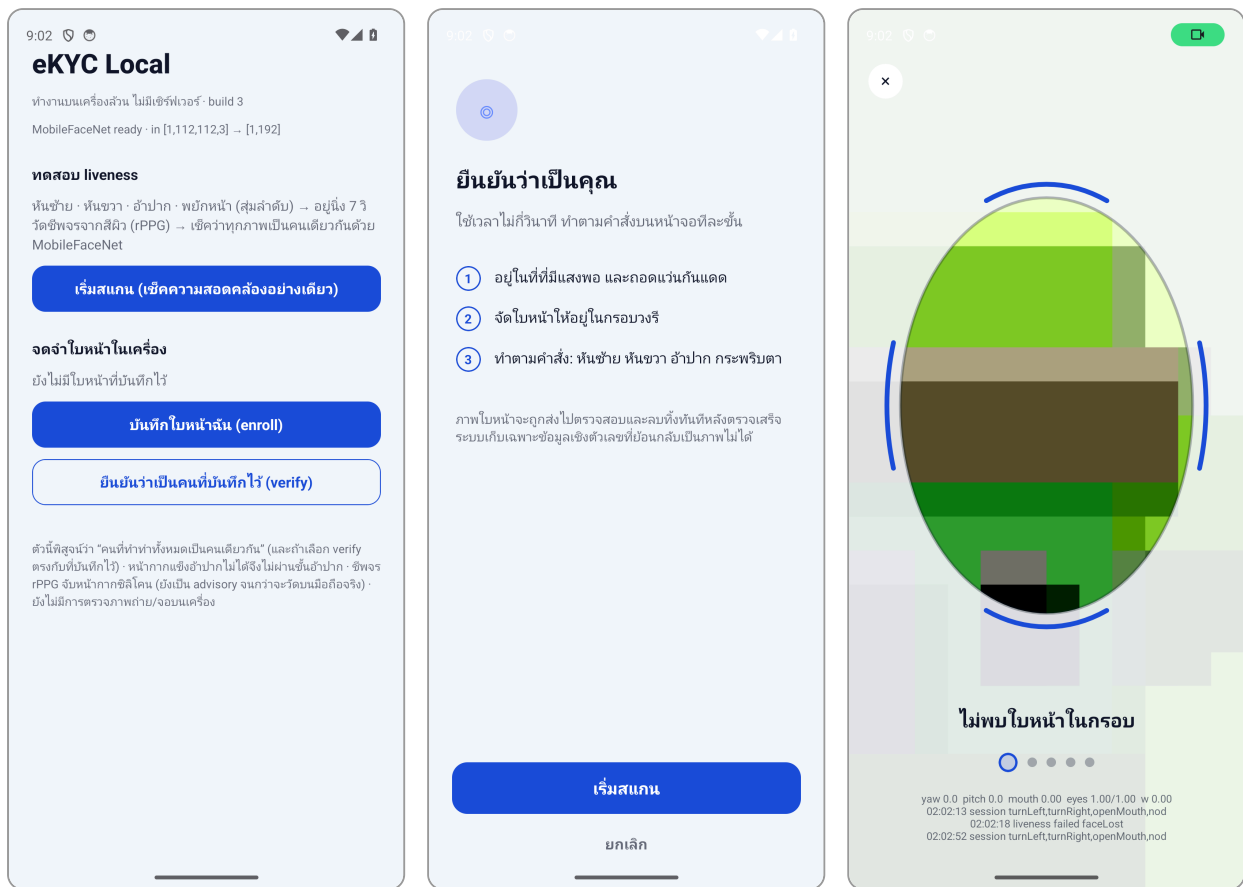
กลุ่ม	จำนวนคู่	มัธยฐาน	p5 / p10	p95 / p99
คนเดียวกัน	600	0.591	0.217 / 0.300	—
คนละคน	780	0.212	—	0.467 / 0.559

ตารางที่ 6 อัตราผิดพลาดตามเกณฑ์ (LFW)

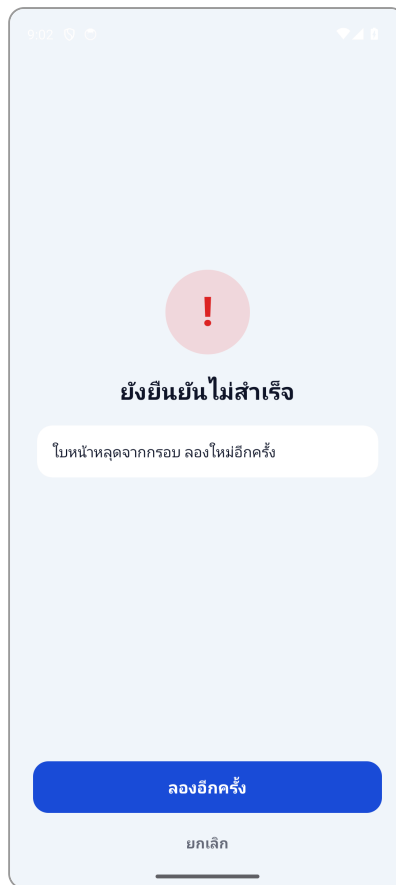
เกณฑ์	ปฏิเสธคนเดียวกัน (FRR)	รับคนละคน (FAR)
0.40	19.0 %	12.05 %
0.45 (consistencyMin)	25.5 %	6.54 %
0.50	33.7 %	3.33 %
0.55 (matchMin)	43.5 %	1.41 %
0.60	51.0 %	0.38 %

หมายเหตุ: ค่า FRR บน LFW สูงเพราะคู่ภาพถ่ายต่างปี ต่างมุม ต่างกล้อง ในการใช้งานจริงภาพทั้งหมดถ่ายภายในเซสชันเดียวกัน ภายในไม่กี่วินาที ค่าคล้ายคลึงจึงสูงกว่านี้มาก ตัวเลขนี้จึงใช้เป็นขอบเขตล่างของสมรรถนะและใช้เลือกเกณฑ์ที่ปลอดภัยฝั่ง FAR

6.3 หน้าจอของแอปพลิเคชัน



รูปที่ 3 หน้าจอแอป eKYC Local (build 3) บน Android Emulator (API 34, x86_64): หน้าแรกแสดง “MobileFaceNet ready · in [1,112,112,3] → [1,192]” ยืนยันว่าโมเดลโหลดบนเครื่องได้จริง, หน้าแนะนำก่อนสแกน, หน้าจับภาพพร้อมกรอบ คำสั่งที่สุ่มได้ (turnLeft, turnRight, openMouth, nod) และค่าจากตัวตรวจจับ (กล้องของ emulator เป็นภาพจำลอง จึงแสดง “ไม่พบใบหน้าในกรอบ”)



รูปที่ 4 หน้าผลลัพธ์เมื่อไม่ผ่าน พร้อมเหตุผลเป็นภาษาไทย (ตัวอย่าง: ใบหน้าหลุดจากกรอบเกินเวลาที่กำหนด —
LOCAL_faceLost)

7. การติดตั้งและการทดสอบด้วยตนเอง

7.1 การติดตั้งไฟล์ APK

1. ดาวน์โหลด `ekyc-local-*.apk` จากหน้า Releases ของ repo
2. เปิดไฟล์บนโทรศัพท์ Android (สถาปัตยกรรม arm64-v8a, Android 8.0 ขึ้นไป) อนุญาตการติดตั้งจากแหล่งที่ไม่รู้จัก
3. เปิดแอป อนุญาตการใช้กล้อง

Release ปัจจุบัน: `local-v1.0.0-b3` (build 3) — ไฟล์ติดตั้งเซ็นด้วย debug keystore สำหรับการทดสอบ ก่อนแจกจ่ายจริงต้องเซ็นด้วยคีย์ของทีม ขนาดไฟล์ประมาณ 59 MB (arm64 เท่านั้น, เปิด ProGuard/shrinkResources) องค์ประกอบหลักคือโมเดล ML Kit ที่ฝังในตัว, TensorFlow Lite runtime, และ React Native/Hermes

7.2 ขั้นตอนทดสอบที่แนะนำ

ตารางที่ 7 กรณีทดสอบ

ลำดับ	กรณี	ผลที่คาดหวัง
1	ปิด Wi-Fi และข้อมูลมือถือ แล้วใช้งานทุกฟังก์ชัน	ทำงานได้ครบ ไม่มีข้อผิดพลาดเครือข่าย (พิสูจน์ว่า local 100 %)
2	“เริ่มสแกน” และทำตามคำสั่งจนครบ	ผ่าน; consistency min โดยทั่วไป ≥ 0.6 ; ลำดับทำต่างกันในแต่ละครั้ง
3	ให้บุคคลที่สองเข้ามาแทนในท่าที่ 2 หรือ 3	ไม่ผ่าน; รหัส <code>IDENTITY_INCONSISTENT</code> ; คู่ที่แย่ที่สุด (weakest) ขึ้นไปที่ท่าที่ถูกสลับ
4	ไม่ทำท่า หรืออ้าปากไม่กว้าง	ขั้นตอนไม่ผ่านภายใน 12 วินาที; รหัส <code>LOCAL_timeout</code>
5	“บันทึกใบหน้าฉัน” แล้ว “ยืนยันว่าเป็นคนที่บันทึกไว้” ด้วยบุคคลเดิม	ผ่าน; match vs saved ≥ 0.55
6	ยืนยันด้วยบุคคลอื่น	ไม่ผ่าน; รหัส <code>NO_MATCH</code>
7	ลบใบหน้าที่ยืนยันไว้ แล้วกด verify	แจ้งให้บันทึกก่อน

บันทึกค่า “consistency min”, “match vs saved” และเวลาที่แสดงบนหน้าผลลัพธ์ของทุกกรณี เพื่อใช้ปรับ `consistencyMin / matchMin` กับกลุ่มผู้ใช้จริง

7.3 การสร้างจากซอร์สโค้ด

```
npm install
npm test # 105 + 16 tests
cd apps/local
npx expo prebuild --platform android
cd android && ./gradlew assembleRelease -PreactNativeArchitectures=arm64-v8a
```

8. ความปลอดภัยและข้อมูลส่วนบุคคล

- **ไม่มีการส่งข้อมูล** — โค้ดของแพ็คเกจและแอปไม่มีการเรียกเครือข่ายใด ๆ ตรวจสอบได้จากซอร์สโค้ดและจากการทดสอบในโหมดไม่มีสัญญาณ
- **ภาพนิ่งชั่วคราว** — ภาพที่ถ่ายระหว่างทำท่าถูกเก็บในแคชชั่วคราวของแอปเพื่อประมวลผล และไม่ได้ถูกส่งไปที่ใด ผู้พัฒนาแอปที่นำไปใช้ควรลบไฟล์ชั่วคราวหลังใช้งาน (แอปตัวอย่างพึ่งพากลไกล้างแคชของระบบ)
- **แม่แบบใบหน้า** — เก็บเป็นตัวเลข 192 ค่าในไฟล์ `ekyc-local-template.json` ในพื้นที่ส่วนตัวของแอป ไม่สามารถแปลงกลับเป็นภาพได้ ผู้ใช้ลบได้จากในแอป
- **ข้อจำกัดด้านการปลอมแปลง** — ดูหัวข้อ 2 และ 9

9. ข้อจำกัดและแผนงานถัดไป

ตารางที่ 8 ข้อจำกัดที่ทราบและแนวทาง

ข้อจำกัด	แนวทางในรุ่นถัดไป
ไม่มีการตรวจสอบสื่อบล็อก 2 มิติ (ภาพถ่าย/จอ) บนอุปกรณ์	เพิ่มโมเดล PAD ขนาดเล็ก (MiniFASNet 1.7 MB แปลงเป็น TFLite) และ/หรือ flash สืบจากหน้าจอแบบเดียวกับระบบ server
rPPG ปรับเทียบบนสัญญาณสังเคราะห์ จึงเป็น advisory	เก็บคะแนนจากผู้ใช้งานจริง ≥ 30 คน และหน้ากาก แล้วตั้ง <code>pulseMin</code> และเปลี่ยนเป็น enforce
เกณฑ์ปรับเทียบจากชุดข้อมูลสาธารณะ ไม่ใช่กลุ่มผู้ใช้งานจริง	เก็บค่าจากหน้าผลลัพธ์ในการทดสอบภาคสนามแล้วปรับ
ยังไม่ได้ทดสอบบนอุปกรณ์จริงหลากหลายรุ่นโดยผู้จัดทำ (ทดสอบผ่านชุดทดสอบอัตโนมัติ, typecheck และ emulator)	รอบทดสอบภาคสนามตามตารางที่ 7
ขนาด APK ประมาณ 59 MB	เปลี่ยน ML Kit เป็นแบบดาวน์โหลดผ่าน Google Play services และตัด GPU delegate ของ TFLite
เซ็นด้วย debug keystore	สร้าง keystore ของทีมและตั้งค่าใน <code>signingConfigs.release</code>

ภาคผนวก ก. รหัสเหตุผล (reason codes)

รหัส	ความหมาย
IDENTITY_INCONSISTENT	คู่ภาพที่แย่มากที่สุดต่ำกว่า consistencyMin — บุคคลไม่ตรงกันระหว่างท่า
NO_MATCH	ภาพ neutral ไม่ตรงกับแม่แบบที่บันทึกไว้ (โหมด verify)
NO_FACE	ML Kit ไม่พบใบหน้าในภาพหนึ่งบางภาพ
FRAME_UNREADABLE	เตรียมภาพ/ฝังใบหน้าไม่สำเร็จ
FRAME_MISSING	จำนวนภาพที่ประมวลผลได้น้อยกว่าจำนวนท่า
PULSE_ABSENT	ไม่พบชีพจร (เฉพาะเมื่อ pulseRule='enforce')
LOCAL_timeout / LOCAL_faceLost / LOCAL_multipleFaces / LOCAL_captureFailed / LOCAL_cancelled	ความล้มเหลวฝังการจับภาพก่อนถึงขั้นตัดสิน

ภาคผนวก ข. โครงสร้างไฟล์ที่เกี่ยวข้อง

ไฟล์	เนื้อหา
packages/react-native-ekyc-local/src/identity.ts	คณิตศาสตร์อัลกอริธึม: cosine, preprocess, decodeJpeg, consistency, judge, เกณฑ์เริ่มต้น
packages/react-native-ekyc-local/src/embedder.ts	โหนดโมเดล TFLite, ตัด/หมุน/ย่อภาพ, ฝังใบหน้า
packages/react-native-ekyc-local/src/LocalLivenessCamera.tsx	หน้าจอจับภาพและการตัดสินบนหน้าจอ
packages/react-native-ekyc-local/src/challenges.ts	ชุดท่าและการสุ่มลำดับ

<code>packages/react-native-ekyc-local/src/pulse.ts</code> , <code>skin.ts</code>	rPPG scorer (TypeScript) และการอ่านสีบริเวณหน้าผาก/แก้ม
<code>packages/react-native-ekyc/src/liveness/challenges.ts</code> , <code>mouth.ts</code>	OpenMouthChallenge, NodChallenge, การวัดการอ้าปาก
<code>packages/react-native-ekyc-local/tests/</code>	ชุดทดสอบ 29 รายการ
<code>apps/local/</code>	แอป Expo สำหรับสร้าง APK (จิมสว่าง เพื่อแยกจากแอปตัวอย่างระบบ server ที่เป็นจิมมืด)

ผู้จัดทำ

ผู้ตรวจสอบ

ผู้อนุมัติ